ViaSP — Issues e sub-issues do projeto

Ponto de Ônibus Inteligente · plano de implementação gerado em 19/10/2026 a partir da documentação do repositório e do backend base

Como usar este documento

- Cada item **ISS-NN** vira uma issue no GitHub e cada **ISS-NN.M** vira uma sub-issue da issue-mãe. Os códigos são provisórios: ao criar as issues, troquem pelos números reais do GitHub nos campos “Depende de”.
- A **ordem do documento é a ordem de implementação**. Uma issue só começa quando todas as listadas em “Depende de” estiverem concluídas, salvo quando o texto indicar que pode andar em paralelo. Dentro de uma issue, cada sub-issue depende da anterior, a menos que se diga o contrário.
- Prioridades seguem o MoSCoW do backlog (Must, Should, Could). Milestones: **E3** (12/11), **E4** (24/11) e **E5** (01/12).
- **Definição de pronto (vale para toda issue):** PR revisado por pelo menos uma pessoa, CI verde, critérios de aceitação verificados e documentação atualizada quando houver mudança de contrato ou de arquitetura.

Premissas e pontos a validar

1. **Linhas do ponto (US01).** A Entrega 1 lista os endpoints de linhas, paradas e previsão, mas nenhum deles devolve diretamente “as linhas que atendem esta parada”. Existe, até onde sei, o endpoint `/Previsao/Parada`, que só lista linhas com veículos previstos naquele momento. Não consegui testar a API real, então a **ISS-03** decide isso antes de qualquer código do cliente.
2. **Entrega 4 (artigo).** O enunciado do repositório é a versão da pós-graduação, então o artigo foi planejado como obrigatório. Se o grupo não tiver integrante de pós, a ISS-38 passa a ser opcional.
3. **Prazo.** De 19/10 a 12/11 são cerca de 3,5 semanas com uma sessão semanal. O MVP (Must) e a documentação de qualidade cabem; acessibilidade (Should) e voz tendem a ficar para depois da E3.
4. **Stack.** Frontend em React/JavaScript e backend em FastAPI, conforme `tecnologias.md`.
5. **Voz/LLM e rotas multimodais** são Could Have e ficam fora do MVP.

Distribuição sugerida por sprint

Sprint	Período	Issues	Entrega
2	19–25/10	ISS-01 a ISS-05 e ISS-14	E3
3	26/10–01/11	ISS-06 a ISS-10, ISS-13, ISS-15 a ISS-17	E3
4	02–08/11	ISS-11, ISS-12, ISS-18 a ISS-23, ISS-28 a ISS-31	E3
5	09–12/11	ISS-32 a ISS-36	E3
6	13–23/11	ISS-24 a ISS-26, ISS-37, ISS-38	E4
7	24/11–01/12	ISS-39 a ISS-42 (e opcionais, se houver tempo)	E5

Caminho crítico: ISS-02 → ISS-03 → ISS-05 → ISS-06 → ISS-07 → ISS-09 → ISS-13 → ISS-18 → ISS-23 → ISS-31 → ISS-36. Atraso em qualquer uma delas atrasa a Entrega 3.

Fase 0 — Preparação do processo e do ambiente

ISS-01 — Configurar o processo Scrum no GitHub

Milestone: E3 · **Prioridade:** Must · **Depende de:** nenhuma

O que fazer: deixar o GitHub pronto para planejar e acompanhar o trabalho como descrito em `docs/gestao/ferramentas-para-gestao.md`, que ainda trata o quadro como "será criado".

Critérios de aceitação:

- As milestones E3, E4 e E5 existem com as datas do enunciado.
- Todas as issues deste documento estão criadas, com labels, milestone e responsável.
- O quadro tem as colunas Backlog, Em andamento, Em revisão e Concluído.

Sub-issues:

- **ISS-01.1 — Criar milestones e labels.** Labels por tipo (`feature`, `bug`, `docs`, `test`, `refactor`), área (`backend`, `frontend`, `docs`), épico (`EP01` a `EP04`), prioridade (`must`, `should`, `could`) e `bloqueada`. *Aceite:* labels visíveis em Issues e três milestones com data.
- **ISS-01.2 — Criar o GitHub Project (Kanban).** Colunas acima e campos Sprint, Prioridade e "Depende de". *Aceite:* quadro vinculado ao repositório e issues entrando nele automaticamente.

- **ISS-01.3 — Templates de issue/PR e CONTRIBUTING.** Template de issue com declaração, critérios de aceitação e “Depende de”; template de PR com “Issue relacionada” e checklist; regras de branch (`feat/iss-12-descricao`), de commit e de revisão (mínimo 1 aprovação); proteger a `main` . *Aceite:* abrir uma issue e um PR de teste mostra os templates e push direto na `main` é bloqueado.
- **ISS-01.4 — Concluir `sprint-1.md` e criar o modelo de sprint.** Preencher milestone e fechamento da Sprint 1 e criar `docs/sprints/_modelo.md` (objetivo, itens, resultado, retrospectiva). *Aceite:* sprint-1 completo e modelo disponível para as próximas.

ISS-02 — Integrar o backend base e preparar o ambiente de desenvolvimento

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-01.3 (pode começar antes, mas o PR segue o fluxo definido ali)

O que fazer: levar o scaffold de `backend/` ao repositório e garantir que roda em todas as máquinas. O scaffold passou apenas por verificação de sintaxe e por um teste manual do domínio e do banco; os testes da API ainda não foram executados.

Critérios de aceitação:

- `pip install -r requirements-dev.txt && pytest` passa em uma máquina limpa de cada integrante.
- `uvicorn app.main:app` sobe e `GET /api/saude` responde 200.
- Nenhum segredo nem arquivo `.db` aparece em `git status` .

Sub-issues:

- **ISS-02.1 — Subir o `backend/` via PR** e referenciar `backend/README.md` no README da raiz. *Aceite:* PR mergeado e README atualizado.
- **ISS-02.2 — Rodar e corrigir os testes do scaffold.** *Aceite:* suíte verde; correções descritas no PR.
- **ISS-02.3 — Obter o token do Olho Vivo e configurar o `.env` .** Cadastro no portal de desenvolvedores da SPTrans; cada integrante cria seu `.env` a partir do `.env.example` . *Aceite:* `.env` ignorado pelo Git, token ausente do histórico e `.env.example` atualizado.
- **ISS-02.4 — Lint e formatação com ruff**, configurados em `pyproject.toml` . *Aceite:* `ruff check` sem erros.
- **ISS-02.5 — CI no GitHub Actions** rodando ruff e pytest a cada PR, sem token real (os testes usam mocks). **Depende de:** ISS-02.2 e ISS-02.4. *Aceite:* PR com teste quebrado fica vermelho e PR correto fica verde.

Fase 1 — Integração com a API Olho Vivo (backend)

ISS-03 — Spike: explorar a API real e fechar as decisões técnicas

Milestone: E3 · **Prioridade:** Must · **Origem:** F05 · **Depende de:** ISS-02.3

O que fazer: validar na API real o que os documentos apenas assumem, gerar fixtures verdadeiras para os testes e registrar as decisões.

Critérios de aceitação:

- Existe `docs/produto/decisoes/ADR-001-linhas-do-ponto.md` com a estratégia escolhida para a US01.
- Há fixtures JSON em `backend/tests/fixtures/` para cada endpoint que será usado.
- O `PARADA_CODIGO` real do totem está definido e documentado.

Sub-issues:

- **ISS-03.1 — Explorar os endpoints** (`Login/Autenticar`, `Linha/Buscar`, `Parada/Buscar`, `Parada/BuscarParadasPorLinha`, `Previsao`, `Previsao/Parada`) com curl ou Postman e salvar as respostas reais, sem o token, como fixtures. Incluir um caso sem previsão. *Aceite:* ao menos uma fixture por endpoint, inclusive resposta vazia.
- **ISS-03.2 — Escolher a parada do totem e achar seu código** via `Parada/Buscar`. **Depende de:** ISS-03.1. *Aceite:* código, nome e coordenadas documentados no README do backend.
- **ISS-03.3 — Decidir como listar as linhas do ponto (US01).** Opções: (a) `/Previsao/Parada`, que só traz linhas com veículo previsto no momento; (b) cruzar `Linha/Buscar` com `BuscarParadasPorLinha`, que exige muitas chamadas; (c) lista estática de linhas do ponto, gerada uma vez e mantida em configuração. Registrar prós, contras e consequências em ADR (por exemplo: uma linha sem veículo previsto aparece ou não na tela inicial?). **Depende de:** ISS-03.1 e ISS-03.2. *Aceite:* ADR aprovado pelo grupo.
- **ISS-03.4 — Documentar os limites da API:** expiração da sessão, formato de erro, latência típica e comportamento fora do horário de operação. **Depende de:** ISS-03.1. *Aceite:* seção “Limites observados” no ADR ou no README do backend.

ISS-04 — Completar os métodos do domínio

Milestone: E3 · **Prioridade:** Must · **Origem:** US06–US08 · **Depende de:** ISS-02 (pode andar em paralelo à ISS-03)

O que fazer: implementar o comportamento puro dos objetos, sem rede: `PrevisaoChegada.calcular_tempo_de_espera`, `ConsultaPrevisao.foi_bem_sucedida` e `possui_previsoes`, e definir o papel do `Totem` no backend.

Critérios de aceitação:

- Nenhum método do domínio lança `NotImplementedError` e todos têm teste unitário.
- A espera nunca é negativa e trata a virada de meia-noite (por exemplo, previsão “00:03” com referência “23:58”).
- Uma consulta com estado `SEM_PREVISAO` é considerada bem-sucedida, mas sem previsões, como no diagrama de classes.

Sub-issues:

- **ISS-04.1 — `calcular_tempo_de_espera`** aceitando referência em `HH:MM` ou `datetime`. *Aceite:* testes de espera normal, zero, meia-noite e previsão no passado.
- **ISS-04.2 — `foi_bem_sucedida` e `possui_previsoes`.** *Aceite:* testes para os três estados de `EstadoConsulta`.
- **ISS-04.3 — Decidir o destino das operações do `Totem`** (`buscar_linha`, `selecionar_linha`, `voltar_ao_inicio`), que no diagrama representam a interação do passageiro e hoje pertencem ao frontend, e atualizar `diagrama-de-classes.md`. *Aceite:* diagrama e código coerentes.

ISS-05 — ClienteSPTrans: autenticação e tratamento de erros

Milestone: E3 · **Prioridade:** Must · **Origem:** F05 · **Depende de:** ISS-03

O que fazer: implementar a sessão com o Olho Vivo e traduzir qualquer falha de comunicação em exceções de `excecoes.py`, base do tratamento de erros da US08.

Critérios de aceitação:

- A primeira chamada autentica sozinha e uma sessão expirada reautentica uma vez e repete a chamada.
- Token vazio ou inválido gera `ErroAutenticacaoSPTrans`; timeout, falha de rede e 5xx geram `SPTransIndisponivel`; o restante gera `ErroSPTrans`.
- O token não aparece em logs nem em mensagens de erro.

Sub-issues:

- **ISS-05.1 — `autenticar()` e método interno `_requisitar`** que garante autenticação, mantém o cookie de sessão do `httpx.AsyncClient` e reautentica quando necessário. *Aceite:* testes com `httpx.MockTransport` para login ok, login recusado e sessão expirada.

- **ISS-05.2 — Tradução de erros HTTP e de rede** em exceções, com timeout configurável. **Depende de:** ISS-05.1. *Aceite:* um teste por tipo de exceção.
- **ISS-05.3 — Mascarar o token** em logs e mensagens. **Depende de:** ISS-05.1. *Aceite:* teste garante que o token não aparece em `str(exceção)` nem nos logs.

ISS-06 — ClienteSPTrans: linhas e paradas

Milestone: E3 · **Prioridade:** Must · **Origem:** F01, F02, F03, F05 · **Depende de:** ISS-05 e ISS-03.3

O que fazer: consultar linhas e paradas e converter o JSON da SPTrans (que usa chaves abreviadas) em objetos de domínio.

Critérios de aceitação:

- As fixtures reais da ISS-03 viram objetos `Linha` e `Parada` corretos.
- Resposta vazia gera lista vazia e campo ausente não derruba o parsing.

Sub-issues:

- **ISS-06.1 —** **`buscar_linhas(termo)`** devolvendo `Linha` (código, letreiro, sentido, origem e destino). *Aceite:* teste com fixture de `Linha/Buscar`.
- **ISS-06.2 —** **`buscar_paradas(codigo_linha)`** devolvendo `Parada` do itinerário. *Aceite:* teste com fixture de `BuscarParadasPorLinha`.
- **ISS-06.3 — Obter as linhas do ponto** conforme o ADR-001, como novo método do cliente. **Depende de:** ISS-03.3. *Aceite:* teste com a fixture escolhida na ADR.
- **ISS-06.4 —** **`buscar_parada(codigo)`** com nome, endereço e coordenadas da parada configurada (insumo da US05). *Aceite:* teste com fixture de `Parada/Buscar`.

ISS-07 — ClienteSPTrans: previsões

Milestone: E3 · **Prioridade:** Must · **Origem:** F06, F07, F08 · **Depende de:** ISS-04 e ISS-05

O que fazer: consultar a previsão de uma linha em uma parada e montar a `ConsultaPrevisao`.

Critérios de aceitação:

- `obter_previsoes` devolve `ConsultaPrevisao` com `horario_referencia` vindo da API (base da US07), veículos e horários previstos.
- Resposta sem veículos gera estado `SEM_PREVISAO`, sem erro.
- Falhas de comunicação propagam as exceções da ISS-05.

Sub-issues:

- **ISS-07.1 — Parsing da resposta de `Previsao`** em `ConsultaPrevisao`, `PrevisaoChegada` e `Veiculo`. *Aceite:* teste com fixture real.
- **ISS-07.2 — Caso sem previsão e campos vazios ou nulos. Depende de:** ISS-07.1. *Aceite:* testes para parada sem veículos e para campos ausentes.

Fase 2 — Serviço, logs e API REST

ISS-08 — ServicoConsulta: buscar linhas do ponto (US01 e US02)

Milestone: E3 · **Prioridade:** Must · **Origem:** US01, US02 · **Depende de:** ISS-06

O que fazer: implementar `buscar_linhas(parada, termo)` usando o cliente: sem termo devolve as linhas do ponto; com termo filtra por número ou nome, ignorando maiúsculas e acentos.

Critérios de aceitação:

- Sem termo, retorna todas as linhas do ponto com sentido, origem e destino.
- A busca por “8000”, “paulista” ou “PAULISTA” retorna o mesmo conjunto relevante.
- Termo sem resultado retorna lista vazia, não erro.

Sub-issues:

- **ISS-08.1 — Listagem das linhas do ponto** conforme o ADR-001. *Aceite:* teste unitário com cliente falso (fake).
- **ISS-08.2 — Filtro por termo** (número, origem e destino, normalizando acentos). **Depende de:** ISS-08.1. *Aceite:* testes com termos válidos, inexistentes e vazios.

ISS-09 — ServicoConsulta: previsões de chegada (US06, US07 e US08)

Milestone: E3 · **Prioridade:** Must · **Origem:** US06, US07, US08 · **Depende de:** ISS-07 e ISS-04

O que fazer: implementar `consultar_previsoes(parada, linha)` devolvendo sempre algo que o totem consiga mostrar. Decisão a registrar no PR: falhas de comunicação viram `ConsultaPrevisao` com estado `FALHA` e uma `mensagem_erro` (campo a adicionar ao domínio e ao schema), em vez de estourar uma exceção opaca.

Critérios de aceitação:

- Sucesso com veículos retorna `SUCESSO`, previsões ordenadas por horário e `horario_consulta` preenchido.
- Sucesso sem veículos retorna `SEM_PREVISAO` com `previsoes` vazio.

- Falha de comunicação retorna `FALHA` com mensagem clara em português, sem detalhes técnicos.

Sub-issues:

- **ISS-09.1 — Fluxo de sucesso e ordenação** das previsões por tempo de espera. *Aceite:* teste unitário.
- **ISS-09.2 — Adicionar `mensagem_erro` ao domínio e ao schema** e converter as exceções da ISS-05 em `FALHA`. **Depende de:** ISS-09.1. *Aceite:* um teste para cada exceção.
- **ISS-09.3 — Registrar `horario_consulta`** com fuso `America/Sao_Paulo`, para a US07 mostrar a hora correta. **Depende de:** ISS-09.1. *Aceite:* teste com relógio injetado.

ISS-10 — ServicoConsulta: itinerário da linha (US03)

Milestone: E3 · **Prioridade:** Must · **Origem:** US03 · **Depende de:** ISS-06

O que fazer: implementar `consultar_paradas(linha)` entregando um itinerário simplificado: destino final e paradas principais.

Critérios de aceitação:

- A resposta traz destino final e uma lista reduzida de paradas principais, não centenas de paradas.
- A regra de “parada principal” está documentada.

Sub-issues:

- **ISS-10.1 — Definir e documentar a regra de paradas principais** (por exemplo: primeira, última e até N intermediárias distribuídas ao longo do trajeto, ou paradas em terminais e corredores). *Aceite:* regra aprovada e escrita no README do backend.
- **ISS-10.2 — Implementar a seleção. Depende de:** ISS-10.1. *Aceite:* testes com linhas curtas, longas e sem paradas.

ISS-11 — Cache em memória e proteção da API externa

Milestone: E3 · **Prioridade:** Should · **Depende de:** ISS-08, ISS-09 e ISS-10

O que fazer: evitar chamadas repetidas à SPTrans quando vários passageiros consultam o mesmo ponto e permitir mostrar o último dado válido se a API cair.

Critérios de aceitação:

- Duas consultas iguais dentro do tempo de vida configurado geram uma só chamada à SPTrans.
- Se a SPTrans falhar e houver resultado anterior, ele é devolvido marcado como desatualizado.

- O tempo de vida é configurável por variável de ambiente.

Sub-issues:

- **ISS-11.1 — Cache com TTL** para linhas e paradas (dados quase estáticos) e TTL curto para previsões. *Aceite:* teste com relógio simulado.
- **ISS-11.2 — Servir o último resultado válido em caso de falha**, com indicador de desatualizado. **Depende de:** ISS-11.1. *Aceite:* teste simulando queda da API.

ISS-12 — Persistência de logs de uso

Milestone: E3 · **Prioridade:** Should · **Origem:** arquitetura (passo 6) · **Depende de:** ISS-09

O que fazer: implementar `RepositorioLogs.registrar_consulta` e gravar, em segundo plano, linha, parada, estado, latência e erro de cada consulta, sem atrasar a resposta ao totem. Esses logs também servem de evidência da integração com a API na Entrega 5.

Critérios de aceitação:

- Cada consulta de previsão gera uma linha em `log_consulta`.
- Falha ao gravar o log não afeta a resposta ao passageiro.
- O token e dados pessoais nunca são gravados.

Sub-issues:

- **ISS-12.1 — Implementar o INSERT** em `registrar_consulta`. *Aceite:* teste com banco temporário.
- **ISS-12.2 — Medir a latência da chamada à SPTrans.** *Aceite:* `latencia_ms` preenchido nos testes.
- **ISS-12.3 — Chamar o repositório via `BackgroundTasks`** nas rotas. **Depende de:** ISS-12.1 e ISS-12.2. *Aceite:* resposta devolvida mesmo com o banco indisponível.

ISS-13 — Finalizar a API REST

Milestone: E3 · **Prioridade:** Must · **Origem:** US01–US08 · **Depende de:** ISS-06.4, ISS-08, ISS-09 e ISS-10

O que fazer: remover os 501 das rotas, congelar o contrato que o frontend vai consumir e padronizar os erros.

Critérios de aceitação:

- As rotas `/api/linhas`, `/api/linhas/{codigo}/paradas`, `/api/previsoes` e `/api/totem` respondem dados reais.
- Toda resposta de erro segue o mesmo formato JSON e indica se vale tentar novamente.
- `/docs` (OpenAPI) descreve todas as rotas e schemas.

Sub-issues:

- **ISS-13.1 — Documentar o contrato** em `docs/produto/api.md`, com exemplos de resposta (insumo para o frontend usar mocks). *Aceite:* documento revisado pela dupla do frontend.
- **ISS-13.2 — Padronizar o erro** (`codigo`, `mensagem`, `tentar_novamente`) nos handlers de `main.py`, substituindo o 501. **Depende de:** ISS-13.1. *Aceite:* testes para 502, 503 e 422.
- **ISS-13.3 — `/api/totem` com dados reais da parada** (nome e coordenadas, para o mapa da US05), carregados na subida. **Depende de:** ISS-06.4. *Aceite:* resposta traz nome, endereço, latitude e longitude.
- **ISS-13.4 — Ligar as rotas aos serviços e ajustar os testes** que hoje esperam 501. **Depende de:** ISS-13.2. *Aceite:* suíte verde sem testes de 501.

Fase 3 — Frontend do MVP (React)

ISS-14 — Estruturar o frontend

Milestone: E3 · **Prioridade:** Must · **Origem:** EP01 · **Depende de:** ISS-02 (pode começar em paralelo à Fase 1; usa mocks até a ISS-13.1 estar pronta)

O que fazer: criar o projeto React em `frontend/`, com a base que todas as telas vão usar. Nada de regra de negócio aqui.

Critérios de aceitação:

- `npm install && npm run dev` sobe a aplicação e `npm test` roda.
- O cliente HTTP chama o backend (ou mocks) usando apenas o contrato documentado.
- Cores, tipografia e espaçamentos vêm de variáveis (design tokens) baseadas no protótipo do Figma.

Sub-issues:

- **ISS-14.1 — Criar o projeto** (Vite + React + JavaScript, como em `tecnologias.md`), ESLint, Prettier e proxy de desenvolvimento para `/api`. *Aceite:* aplicação inicial renderiza e lint passa.
- **ISS-14.2 — Organizar pastas e a navegação** por estado entre as telas (Início e Detalhe da linha), sem rotas de URL, já que o totem não tem barra de endereço. **Depende de:** ISS-14.1. *Aceite:* trocar de tela por estado, com botão de voltar sempre visível.
- **ISS-14.3 — Cliente da API e mocks** seguindo o contrato da ISS-13.1 (por exemplo, MSW ou arquivos JSON), com tratamento central de erro no formato padrão. **Depende de:** ISS-14.1 e ISS-13.1. *Aceite:* telas conseguem trabalhar sem o backend.

- **ISS-14.4 — Design tokens e componentes base** (`Botao`, `Cartao`, `TextoGrande`) a partir do Figma, com alvos de toque grandes (sugestão: mínimo de 64 px) e contraste mínimo AA. **Depende de:** ISS-14.1. *Aceite:* componentes documentados em uma página de demonstração.
- **ISS-14.5 — Adicionar lint e testes do frontend ao CI. Depende de:** ISS-14.1 e ISS-02.5. *Aceite:* PR que quebra o frontend fica vermelho.

ISS-15 — Tela inicial: linhas do ponto e sentidos (US01)

Milestone: E3 · **Prioridade:** Must · **Origem:** US01 / F01 · **Depende de:** ISS-14.2, ISS-14.3 e ISS-14.4 (dados reais após a ISS-13)

O que fazer: mostrar, ao abrir o totem, a lista de linhas do ponto com o sentido de cada uma, pensada para a Fátima: poucos elementos, texto grande e leitura imediata.

Critérios de aceitação (da US01):

- Dado que o totem está na tela inicial, quando a passageira olha a tela, então vê a lista das linhas do ponto com número, destino e sentido.
- Textos com tamanho mínimo definido nos tokens e contraste AA.
- Há estados de carregando, lista vazia e erro (o tratamento completo do erro vem na ISS-20).

Sub-issues:

- **ISS-15.1 — Componente `ItemLinha`** (número, destino, sentido) como botão grande. *Aceite:* teste de renderização com dados de exemplo.
- **ISS-15.2 — Tela `Inicio`** buscando as linhas via API e tratando lista longa com rolagem por toque. **Depende de:** ISS-15.1. *Aceite:* lista aparece com mock e com backend real.
- **ISS-15.3 — Estado de carregando e de lista vazia** com mensagem clara. **Depende de:** ISS-15.2. *Aceite:* mensagem “Nenhuma linha encontrada” quando a API devolve lista vazia.

ISS-16 — Busca por toque e teclado virtual (US02)

Milestone: E3 · **Prioridade:** Must · **Origem:** US02 / F02 · **Depende de:** ISS-15

O que fazer: permitir filtrar as linhas por número ou nome com um teclado virtual próprio, pois o totem em modo quiosque não tem teclado físico nem teclado do sistema.

Critérios de aceitação (da US02):

- Dado que o passageiro está na tela inicial, quando digita no teclado virtual, então a lista é filtrada em tempo real.
- Ao tocar em uma linha, abre-se o detalhe dela.

- Há tecla de apagar e de limpar a busca, com alvos de toque grandes.

Sub-issues:

- **ISS-16.1 — Componente `TecladoVirtual`** com teclas numéricas e alfabéticas, apagar e limpar. *Aceite:* testes de digitação e remoção.
- **ISS-16.2 — Campo de busca ligado à API** (`/api/linhas?termo=`) com atraso curto (debounce) para não sobrecarregar o backend. **Depende de:** ISS-16.1 e ISS-13. *Aceite:* uma chamada por pausa de digitação.
- **ISS-16.3 — Seleção do resultado** levando ao detalhe da linha. **Depende de:** ISS-16.2. *Aceite:* tocar em um resultado muda de tela.

ISS-17 — Detalhe da linha: itinerário simplificado (US03)

Milestone: E3 · **Prioridade:** Must · **Origem:** US03 / F03 · **Depende de:** ISS-15, ISS-10 e ISS-13

O que fazer: ao escolher uma linha, mostrar o destino final e as paradas principais para a passageira confirmar que o ônibus serve.

Critérios de aceitação (da US03):

- Dado que o passageiro escolheu uma linha, quando a tela de detalhes carrega, então ela exibe o destino final e as paradas principais.
- Há botão grande de voltar à lista.
- A tela de detalhe é o ponto de entrada das previsões (ISS-18).

Sub-issues:

- **ISS-17.1 — Tela `DetalheLinha`** com cabeçalho (número, sentido, destino). *Aceite:* teste de renderização.
- **ISS-17.2 — Lista de paradas principais** vinda de `/api/linhas/{codigo}/paradas`. **Depende de:** ISS-17.1. *Aceite:* lista exibida com dados reais.

ISS-18 — Previsões de chegada e última atualização (US06 e US07)

Milestone: E3 · **Prioridade:** Must · **Origem:** US06, US07 / F06, F07 · **Depende de:** ISS-17 e ISS-09/ISS-13

O que fazer: mostrar, na tela de detalhe, quanto falta para os próximos ônibus, atualizando sozinha e informando a hora da última atualização.

Critérios de aceitação:

- Os próximos ônibus aparecem com tempo de espera em minutos e horário previsto, de forma clara e contínua (US06).
- A tela informa o horário da última atualização da previsão (US07).

- Se a informação estiver desatualizada além do limite configurado, a tela avisa isso.
- Ao sair da tela, a atualização automática é cancelada.

Sub-issues:

- **ISS-18.1 — Lista de previsões** (minutos de espera em destaque e horário previsto). *Aceite:* teste com previsões, e com lista vazia exibindo “Sem previsão no momento”.
- **ISS-18.2 — Atualização automática (polling)** a cada N segundos (configurável), cancelada ao sair da tela. **Depende de:** ISS-18.1. *Aceite:* teste com temporizador simulado.
- **ISS-18.3 — Contagem regressiva local** entre uma atualização e outra, para a espera não ficar parada. **Depende de:** ISS-18.2. *Aceite:* o valor diminui sem nova chamada e é corrigido a cada resposta.
- **ISS-18.4 — Indicador de última atualização** com horário de referência e aviso de dado desatualizado. **Depende de:** ISS-18.2 e ISS-09.3. *Aceite:* horário visível e alerta quando passa do limite.

ISS-19 — Mapa com a localização do ponto (US05)

Milestone: E3 · **Prioridade:** Must · **Origem:** US05 · **Depende de:** ISS-14 e ISS-13.3

O que fazer: exibir de forma contínua um mapa dos arredores com a posição do ponto marcada, para a passageira se situar.

Critérios de aceitação (da US05):

- A posição do ponto aparece clara no mapa, com marcador grande e legível.
- O mapa permanece visível nas telas principais (por exemplo, como faixa fixa ou painel lateral).
- Se o mapa não carregar, a tela mostra o endereço do ponto em texto e o restante continua funcionando.

Sub-issues:

- **ISS-19.1 — Escolher a biblioteca e o provedor de mapa** (por exemplo, Leaflet com tiles do OpenStreetMap), verificando licença e limites de uso. *Aceite:* decisão registrada no PR.
- **ISS-19.2 — Componente `MapaDoPonto`** centrado nas coordenadas de `/api/totem`, com zoom fixo e sem gestos que tirem o ponto de vista. **Depende de:** ISS-19.1. *Aceite:* marcador visível em teste manual.
- **ISS-19.3 — Fallback quando o mapa falha** (endereço em texto). **Depende de:** ISS-19.2. *Aceite:* simular falha de rede mostra o endereço.

ISS-20 — Falhas, indisponibilidade e nova tentativa (US08)

Milestone: E3 · **Prioridade:** Must · **Origem:** US08 / F08 · **Depende de:** ISS-18 e ISS-13.2

O que fazer: tratar na interface todas as falhas possíveis, comunicando com calma e clareza e oferecendo “Tentar novamente”.

Critérios de aceitação (da US08):

- Cobertura das falhas: backend fora do ar, SPTrans indisponível, autenticação, tempo esgotado e resposta sem previsão.
- Cada falha tem mensagem curta em linguagem simples, sem termos técnicos.
- O botão “Tentar novamente” refaz a consulta e some quando a falha passa.
- Se houver dado anterior, ele continua visível, marcado como desatualizado.

Sub-issues:

- **ISS-20.1 — Catálogo de mensagens de erro** por tipo de falha, revisado quanto à clareza (idealmente lido por alguém fora do grupo). *Aceite:* tabela falha → mensagem em `docs/produto/`.
- **ISS-20.2 — Componente `AvisoErro`** com botão de nova tentativa. **Depende de:** ISS-20.1. *Aceite:* teste de clique refazendo a chamada.
- **ISS-20.3 — Integrar nas telas Inicio e DetalheLinha**, mantendo o último dado válido. **Depende de:** ISS-20.2. *Aceite:* testes simulando cada falha.

ISS-21 — Retorno automático à tela inicial por inatividade (US04)

Milestone: E3 · **Prioridade:** Should · **Origem:** US04 / F04 · **Depende de:** ISS-15 e ISS-17

O que fazer: devolver o totem à tela inicial depois de um período sem interação, limpando busca e seleção, para proteger a privacidade e deixar o equipamento pronto para a próxima pessoa.

Critérios de aceitação (da US04):

- Dado que o totem mostra o detalhe de uma busca anterior, quando passa o tempo de inatividade configurado, então volta à tela inicial.
- Qualquer toque reinicia a contagem.
- A busca digitada e a linha selecionada são apagadas.
- Um aviso curto antes do retorno evita surpresa para quem lê devagar.

Sub-issues:

- **ISS-21.1 — Hook `useInatividade`** com tempo configurável (sugestão inicial: 60 s). *Aceite:* testes com temporizador simulado.

- **ISS-21.2 — Aviso de retorno iminente e reset do estado. Depende de:** ISS-21.1. *Aceite:* aviso aparece nos últimos segundos e o estado é limpo.

ISS-22 — Modo quiosque e execução do conjunto

Milestone: E3 · **Prioridade:** Must · **Origem:** `tecnologias.md` · **Depende de:** ISS-20 e ISS-21

O que fazer: deixar o sistema pronto para rodar no totem: backend e frontend juntos, navegador em tela cheia e bloqueado.

Critérios de aceitação:

- Um único comando (ou script) sobe backend e frontend de produção.
- O navegador abre em tela cheia, sem barra de endereço, e o passageiro não consegue sair da aplicação nem abrir outras páginas.
- Após reinício do equipamento, a aplicação volta sozinha.

Sub-issues:

- **ISS-22.1 — Build de produção servido pelo FastAPI** (arquivos estáticos) ou por proxy, com o frontend e a API na mesma origem. *Aceite:* abrir `http://localhost:8000` mostra o totem completo.
- **ISS-22.2 — Script de inicialização e modo quiosque** (Chromium com `--kiosk`, desativar zoom, menu de contexto e seleção de texto). **Depende de:** ISS-22.1. *Aceite:* roteiro de teste manual executado em tela touch.
- **ISS-22.3 — Documentar a instalação do totem** em `docs/produto/instalacao.md`, incluindo variáveis de ambiente. **Depende de:** ISS-22.2. *Aceite:* outra pessoa do grupo reproduz a instalação só com o documento.

ISS-23 — Integração ponta a ponta do MVP

Milestone: E3 · **Prioridade:** Must · **Origem:** US01–US08 · **Depende de:** ISS-15 a ISS-22 e ISS-13

O que fazer: juntar tudo sem mocks, usando a API real, e validar cada critério de aceitação das US01 a US08. Esta é a marca de “MVP funcional” e o gatilho para a fase de qualidade.

Critérios de aceitação:

- Cada critério de aceitação de US01 a US08 foi executado no totem real ou no ambiente completo e registrado como passou ou falhou.
- Os defeitos encontrados viraram issues com label `bug`.
- Há uma demonstração completa: consultar linha, ver itinerário, ver previsão, forçar falha e tentar de novo.

Sub-issues:

- **ISS-23.1 — Remover os mocks do frontend** da configuração de produção. *Aceite:* build de produção sem MSW.
- **ISS-23.2 — Roteiro de validação manual** com uma linha por critério de aceitação. **Depende de:** ISS-23.1. *Aceite:* planilha ou tabela preenchida, anexada à issue.
- **ISS-23.3 — Corrigir os defeitos críticos encontrados. Depende de:** ISS-23.2. *Aceite:* nenhum bug crítico aberto para US01–US08.

Fase 4 — Acessibilidade e voz (Should e Could)

ISS-24 — Alto contraste e tipografia ampliada (US09)

Milestone: E4 · **Prioridade:** Should · **Origem:** US09 / F09 · **Depende de:** ISS-23

O que fazer: oferecer um botão de acessibilidade visual que troca para um tema de alto contraste e aumenta o texto em toda a interface, sem quebrar o layout.

Critérios de aceitação (da US09):

- Dado que o totem mostra a interface padrão, quando a passageira ativa a opção, então o tema muda para alto contraste e todo texto aumenta.
- Contraste mínimo AAA nos textos principais no tema alto contraste.
- O modo volta ao padrão na inatividade (ISS-21).

Sub-issues:

- **ISS-24.1 — Segundo conjunto de tokens** (alto contraste) e alternância de tema por classe no elemento raiz. *Aceite:* alternância visível em todas as telas.
- **ISS-24.2 — Escala de fonte ampliada** aplicada a todos os textos, com revisão do layout para não cortar conteúdo. **Depende de:** ISS-24.1. *Aceite:* nenhuma tela com texto cortado ou sobreposto.
- **ISS-24.3 — Botão de acessibilidade sempre visível** e resetado pela inatividade. **Depende de:** ISS-24.2 e ISS-21. *Aceite:* o botão aparece em todas as telas e o modo reinicia ao voltar ao início.

ISS-25 — Navegação que não depende só de cor (US10)

Milestone: E4 · **Prioridade:** Should · **Origem:** US10 / F10 · **Depende de:** ISS-24

O que fazer: auditar toda a interface e garantir que estados, alertas e botões usem também ícones, formas e texto.

Critérios de aceitação (da US10):

- Nenhuma informação (alerta, desatualizado, erro, selecionado) é transmitida somente por cor.

- Verificação com simulador de daltonismo em todas as telas, sem perda de informação.

Sub-issues:

- **ISS-25.1 — Auditoria das telas** com simulador de daltonismo, listando os pontos que dependem só de cor. *Aceite:* lista de problemas registrada.
- **ISS-25.2 — Corrigir com ícones, padrões e rótulos. Depende de:** ISS-25.1. *Aceite:* nova rodada da auditoria sem pendências.

ISS-26 — Leitura em áudio das informações (US11)

Milestone: E4 · **Prioridade:** Should · **Origem:** US11 / F11 · **Depende de:** ISS-23

O que fazer: ler em voz alta as opções e as previsões, para o Kleber usar o totem sem enxergar a tela. A síntese pode usar a Web Speech API do navegador, que não precisa de serviço externo.

Critérios de aceitação (da US11):

- Dado que o totem está na tela inicial ou no detalhe, quando o usuário ativa o áudio, então o sistema lê os textos e as opções selecionadas.
- A ativação não depende de achar um botão na tela: definir e documentar um controle acessível (por exemplo, botão físico ou uma faixa fixa e grande na borda da tela).
- A leitura pode ser interrompida e repetida, e a previsão é lida em frases curtas (“Linha 8000, chega em 4 minutos”).
- Se não houver voz em português instalada no sistema, o totem avisa a equipe nos logs e mostra um aviso.

Sub-issues:

- **ISS-26.1 — Definir como o usuário cego ativa o áudio** (controle físico ou área da tela) e validar com o grupo e, se possível, com uma pessoa com deficiência visual. *Aceite:* decisão documentada.
- **ISS-26.2 — Serviço de fala** (`speechSynthesis` , pt-BR) com fila, interrupção e repetição. **Depende de:** ISS-26.1. *Aceite:* testes com a API simulada.
- **ISS-26.3 — Ler a lista de linhas, o detalhe e as previsões** em frases curtas e naturais. **Depende de:** ISS-26.2. *Aceite:* roteiro manual cobrindo as três telas.
- **ISS-26.4 — Verificar a voz em português no equipamento do totem** e tratar a ausência. **Depende de:** ISS-26.2. *Aceite:* aviso exibido quando não há voz pt-BR.

ISS-27 — Interação por comandos de voz (US12)

Milestone: E5 · **Prioridade:** Could · **Origem:** US12 / F12 · **Depende de:** ISS-26 (fazer apenas se o MVP e a ISS-26 estiverem estáveis)

O que fazer: permitir falar um comando (“buscar linha para o centro”) e receber a resposta em áudio e na tela. Segue a arquitetura opcional: captura de áudio no frontend, transcrição e extração de intenção no backend.

Critérios de aceitação (da US12):

- Um comando de voz válido gera a busca e a resposta em áudio e na tela.
- Falha de transcrição ou de interpretação gera mensagem clara e permite repetir.
- O áudio não é armazenado e as chaves das APIs ficam só em variáveis de ambiente.
- O sistema inteiro continua funcionando se as APIs de voz estiverem fora do ar.

Sub-issues:

- **ISS-27.1 — Decisão e custo:** escolher o serviço de transcrição e a LLM, estimar custo e limites e registrar em ADR. *Aceite:* ADR aprovada.
- **ISS-27.2 — Captura de áudio no frontend** com Web Audio API e permissão de microfone. **Depende de:** ISS-27.1. *Aceite:* envio de um trecho de áudio ao backend.
- **ISS-27.3 — Endpoint `/api/voz` no backend:** transcrição e extração de intenção (linha ou destino). **Depende de:** ISS-27.1. *Aceite:* testes com os provedores simulados.
- **ISS-27.4 — Fluxo completo e tratamento de erros. Depende de:** ISS-27.2 e ISS-27.3. *Aceite:* demonstração de ponta a ponta com e sem falha.

Fase 5 — Qualidade e documentação da Entrega 3 (12/11)

O enunciado exige arquitetura documentada, plano e relatório de testes (unitários, integração e/ou aceitação) e relatório de refatorações. Os itens abaixo começam em paralelo às fases anteriores (marcado em cada um) e são fechados depois do MVP integrado.

ISS-28 — Plano de testes

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-13 (pode começar antes; fechar após a ISS-23)

O que fazer: escrever `docs/qualidade/plano-de-testes.md` dizendo o que será testado, como, com quais ferramentas e quais critérios de saída.

Critérios de aceitação:

- Cada critério de aceitação das US01 a US08 (e das US09 a US11, se implementadas) está ligado a pelo menos um teste planejado.
- O plano define níveis (unitário, integração, aceitação), ferramentas (pytest, Vitest, Playwright), ambientes e dados (fixtures reais da ISS-03).
- Define a meta de cobertura e a política para a API real nos testes.

Sub-issues:

- **ISS-28.1 — Matriz de rastreabilidade US → critério → teste.** *Aceite:* tabela completa, sem critério sem teste.
- **ISS-28.2 — Estratégia, ferramentas, ambientes e metas. Depende de:** ISS-28.1. *Aceite:* seções revisadas pelo grupo.

ISS-29 — Testes unitários e de integração do backend

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-13 e ISS-28.2 (o código de teste de cada módulo deve nascer junto com ele, nas ISS-04 a ISS-13)

O que fazer: consolidar e completar a suíte de testes do backend, com medição de cobertura.

Critérios de aceitação:

- Cobertura mínima de 80% em `dominio` e `servicos` (ajustar a meta no plano, se necessário).
- Os testes de integração exercitam as rotas com `TestClient` e a SPTrans simulada com as fixtures reais.
- Nenhum teste depende de rede ou de token no CI.

Sub-issues:

- **ISS-29.1 — Configurar `pytest-cov`** e publicar o relatório no CI. *Aceite:* cobertura aparece no PR.
- **ISS-29.2 — Completar testes unitários** de domínio, serviços, cliente e repositório. **Depende de:** ISS-29.1. *Aceite:* meta de cobertura atingida.
- **ISS-29.3 — Testes de integração das rotas** (sucesso, sem previsão, cada tipo de falha). **Depende de:** ISS-29.2. *Aceite:* um teste por cenário dos critérios da US08.
- **ISS-29.4 — Teste de contrato com a API real**, marcado (`@pytest.mark.integration`), rodado manualmente com token e fora do CI. **Depende de:** ISS-29.2. *Aceite:* execução registrada, usada como evidência da integração.

ISS-30 — Testes do frontend

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-21 e ISS-28.2

O que fazer: testar componentes e fluxos com Vitest e Testing Library.

Critérios de aceitação:

- Telas Início, Detalhe, teclado virtual, aviso de erro e inatividade têm testes de comportamento.
- Os testes usam os mocks do contrato e rodam no CI.

Sub-issues:

- **ISS-30.1 — Testes de componentes e hooks** (`TecladoVirtual`, `useInatividade`, `AvisoErro`, `ItemLinha`). *Aceite:* testes verdes no CI.
- **ISS-30.2 — Testes das telas** com a API simulada, incluindo falha e lista vazia. **Depende de:** ISS-30.1. *Aceite:* cenários das US01 a US03 e US08 cobertos.

ISS-31 — Testes de aceitação ponta a ponta

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-23, ISS-28 e ISS-30

O que fazer: automatizar, com Playwright, os cenários Dado/Quando/Então das user stories, usando um backend com a SPTrans simulada para ficar determinístico.

Critérios de aceitação:

- Há pelo menos um teste por critério de aceitação das US01 a US08.
- Os testes rodam localmente com um comando e, se possível, no CI.
- Os nomes dos testes citam o ID da US.

Sub-issues:

- **ISS-31.1 — Configurar Playwright e um backend de teste** com SPTrans simulada. *Aceite:* um teste simples passa.
- **ISS-31.2 — Cenários das US01 a US04. Depende de:** ISS-31.1. *Aceite:* testes verdes.
- **ISS-31.3 — Cenários das US05 a US08**, incluindo falha com nova tentativa. **Depende de:** ISS-31.1. *Aceite:* testes verdes.

ISS-32 — Relatório de testes

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-29, ISS-30 e ISS-31

O que fazer: documentar em `docs/qualidade/relatorio-de-testes.md` os testes realizados e seus resultados.

Critérios de aceitação:

- Traz resultados reais (quantidade de testes, aprovados e reprovados, cobertura) por nível: unitário, integração e aceitação.
- Lista defeitos encontrados, correções e pendências.
- Referencia a matriz da ISS-28.1 com o status de cada critério.

Sub-issues:

- **ISS-32.1 — Coletar métricas e logs de execução** (saída do pytest, do Vitest e do Playwright). *Aceite:* números anexados ao relatório.

- **ISS-32.2 — Redigir o relatório** com análise dos resultados. **Depende de:** ISS-32.1. *Aceite:* revisado por duas pessoas.

ISS-33 — Refatoração e relatório de refatorações

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-23 (registro contínuo desde a primeira refatoração)

O que fazer: melhorar a estrutura do código sem mudar o comportamento, protegidos pelos testes, e documentar cada refatoração em `docs/qualidade/relatorio-de-refatoracoes.md`.

Critérios de aceitação:

- Cada refatoração tem motivo (por exemplo, duplicação, nome ruim, função longa), antes e depois e o PR correspondente.
- Os testes continuam verdes em todas elas.
- Não restam `NotImplementedError`, código morto nem TODOs sem issue.

Sub-issues:

- **ISS-33.1 — Registrar as refatorações durante o desenvolvimento** (label `refactor` e uma linha no relatório a cada PR). *Aceite:* relatório atualizado a cada sprint.
- **ISS-33.2 — Varredura final de code smells** (duplicação, funções longas, nomes, tipagem). **Depende de:** ISS-23. *Aceite:* ao menos três refatorações relevantes documentadas.
- **ISS-33.3 — Limpeza** de código morto, TODOs e dependências não usadas. **Depende de:** ISS-33.2. *Aceite:* ruff e ESLint sem avisos.

ISS-34 — Arquitetura completa e documentada

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-23

O que fazer: atualizar `arquitetura.md` para descrever o sistema como foi construído, não só o planejado.

Critérios de aceitação:

- Descreve componentes reais, fluxo de dados, decisões e justificativas.
- O diagrama de classes reflete o código final.
- Um novo integrante entende como rodar e onde mexer só com os documentos.

Sub-issues:

- **ISS-34.1 — Atualizar fluxo e diagrama de sequência** (cache, logs e tratamento de falha incluídos; voz só se feita). *Aceite:* diagrama coerente com o código.

- **ISS-34.2 — Atualizar o diagrama de classes** com as classes finais. *Aceite:* nomes e relações iguais aos do código.
- **ISS-34.3 — Diagrama de componentes e implantação do totem** (hardware, navegador em quiosque, backend, SQLite e SPTrans). *Aceite:* diagrama publicado em `docs/produto/`.
- **ISS-34.4 — Reunir as decisões** (ADR-001 e demais) e o contrato da API. *Aceite:* índice de decisões em `docs/produto/decisoes/`.
- **ISS-34.5 — Rastreabilidade US → código → teste. Depende de:** ISS-28.1. *Aceite:* tabela mapeando cada US a módulos e testes.

ISS-35 — Plano de manutenção e evolução

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-34

O que fazer: escrever `docs/produto/manutencao-e-evolucao.md`, exigido pelo enunciado.

Critérios de aceitação:

- Cobre manutenção corretiva, adaptativa (mudanças na API da SPTrans) e evolutiva.
- Define como trocar token, atualizar dependências e monitorar falhas pelos logs.
- Lista a evolução por prioridade (acessibilidade, voz, rotas, vários pontos) e riscos conhecidos.

Sub-issues:

- **ISS-35.1 — Rotinas de manutenção e operação.** *Aceite:* checklist operacional.
- **ISS-35.2 — Roteiro de evolução e riscos.** *Aceite:* itens ligados ao backlog (F09 a F15).

ISS-36 — Fechamento da Entrega 3

Milestone: E3 · **Prioridade:** Must · **Depende de:** ISS-32, ISS-33, ISS-34 e ISS-35

O que fazer: juntar tudo, conferir o enunciado item por item e entregar até 12/11.

Critérios de aceitação:

- `docs/entregas/entrega-3.md` lista cada requisito da entrega com link e o checklist está completo.
- O README da raiz aponta para os novos documentos e explica como rodar o projeto.
- Existe a tag `v0.3.0` na `main`, com o código atualizado.
- A sprint review está registrada em `docs/sprints/`.

Sub-issues:

- **ISS-36.1 — Criar `entrega-3.md`** no mesmo formato da Entrega 2. *Aceite:* todos os itens do enunciado mapeados.
- **ISS-36.2 — Atualizar README e documentação de sprint.** *Aceite:* links funcionando.
- **ISS-36.3 — Revisão cruzada e tag `v0.3.0`. Depende de:** ISS-36.1 e ISS-36.2. *Aceite:* duas pessoas revisaram e a tag foi criada.

Fase 6 — Avaliação e Entrega 4 (24/11)

ISS-37 — Avaliação da solução com usuários

Milestone: E4 · **Prioridade:** Should · **Depende de:** ISS-23 (de preferência depois das ISS-24 a ISS-26)

O que fazer: coletar evidências para a seção de avaliação do artigo e para o relatório final: usabilidade com pessoas reais e métricas técnicas.

Critérios de aceitação:

- Pelo menos 5 participantes, incluindo alguém próximo ao perfil da Fátima, executam as tarefas principais.
- Há dados de tempo para concluir, taxa de sucesso e uma escala de satisfação (por exemplo, SUS).
- Há métricas técnicas extraídas dos logs (latência da SPTrans e taxa de falha).
- Os participantes assinam consentimento e nenhum dado pessoal é guardado.

Sub-issues:

- **ISS-37.1 — Protocolo do teste:** tarefas, roteiro, questionário e termo de consentimento. *Aceite:* documento aprovado pelo grupo.
- **ISS-37.2 — Aplicar o teste e registrar os resultados. Depende de:** ISS-37.1. *Aceite:* dados consolidados em planilha.
- **ISS-37.3 — Extrair as métricas técnicas dos logs. Depende de:** ISS-12. *Aceite:* tabela de latência e falhas.
- **ISS-37.4 — Analisar os resultados e registrar melhorias. Depende de:** ISS-37.2 e ISS-37.3. *Aceite:* análise escrita e melhorias abertas como issues.

ISS-38 — Artigo científico (template ACM)

Milestone: E4 · **Prioridade:** Must se houver integrante de pós-graduação · **Depende de:** ISS-23 (seção de avaliação depende da ISS-37)

O que fazer: escrever o artigo em duas colunas, de 10 a 12 páginas, no template ACM, com as oito seções obrigatórias.

Critérios de aceitação:

- Segue o template ACM, usa duas colunas e tem entre 10 e 12 páginas.
- Contém: Introdução, Background, Trabalhos Relacionados, Proposta Desenvolvida, Avaliação da Solução, Discussão, Ameaças à Validade e Conclusão.
- Todas as afirmações sobre a avaliação se apoiam em dados da ISS-37.
- Referências completas e PDF final entregue até 24/11.

Sub-issues:

- **ISS-38.1 — Configurar o template ACM** (por exemplo, no Overleaf) e o esqueleto das oito seções. *Aceite:* PDF compila.
- **ISS-38.2 — Introdução e Background** (mobilidade em São Paulo, acessibilidade, Olho Vivo). **Depende de:** ISS-38.1.
- **ISS-38.3 — Trabalhos Relacionados:** pesquisar e comparar pelo menos 8 trabalhos (totens informativos, aplicativos de transporte, acessibilidade em pontos de ônibus). **Depende de:** ISS-38.1.
- **ISS-38.4 — Proposta Desenvolvida** (problema, arquitetura, decisões, telas). **Depende de:** ISS-34.
- **ISS-38.5 — Avaliação da Solução** (método, resultados e análise). **Depende de:** ISS-37.4.
- **ISS-38.6 — Discussão e Ameaças à Validade** (amostra pequena, perfil dos participantes, API externa). **Depende de:** ISS-38.5.
- **ISS-38.7 — Conclusão, resumo e referências. Depende de:** ISS-38.4 a ISS-38.6.
- **ISS-38.8 — Revisão final e conferência do formato. Depende de:** ISS-38.7. *Aceite:* duas revisões, páginas dentro do limite e PDF entregue.

Aceite de cada sub-issue 38.2 a 38.7: seção escrita, com referências, revisada por pelo menos uma pessoa.

Fase 7 — Entrega 5 (01/12)

ISS-39 — Evidência da integração com a API

Milestone: E5 · **Prioridade:** Must · **Depende de:** ISS-12 e ISS-29.4

O que fazer: reunir provas claras de que o sistema consome a API do Olho Vivo de verdade, como o enunciado pede.

Critérios de aceitação:

- Há um registro de uma consulta real, da requisição ao resultado na tela, com horário.
- Há trechos de log e métricas da tabela `log_consulta`.

- O material cabe em um slide e em uma seção do relatório final, sem expor o token.

Sub-issues:

- **ISS-39.1 — Capturar evidências** (logs, capturas de tela e gravação curta). *Aceite:* arquivos em `docs/evidencias/` sem segredos.
- **ISS-39.2 — Montar a tabela de endpoints usados** e quais telas dependem de cada um. **Depende de:** ISS-39.1. *Aceite:* tabela pronta para o relatório e os slides.

ISS-40 — Relatório final

Milestone: E5 · **Prioridade:** Must · **Depende de:** ISS-37 e ISS-33

O que fazer: escrever o relatório com análise dos resultados, desafios superados e melhorias futuras.

Critérios de aceitação:

- Contém resultados (testes e avaliação com usuários), desafios reais encontrados e melhorias futuras ligadas ao backlog.
- Inclui a contribuição de cada integrante.
- Referencia as evidências da ISS-39.

Sub-issues:

- **ISS-40.1 — Análise dos resultados e desafios.** *Aceite:* seções redigidas com dados.
- **ISS-40.2 — Melhorias futuras e lições aprendidas. Depende de:** ISS-40.1. *Aceite:* itens priorizados.
- **ISS-40.3 — Revisão e versão final em PDF. Depende de:** ISS-40.2 e ISS-39.2. *Aceite:* revisado por duas pessoas.

ISS-41 — Apresentação e demonstração ao vivo

Milestone: E5 · **Prioridade:** Must · **Depende de:** ISS-39 e ISS-40

O que fazer: preparar os slides, o roteiro da demonstração e um plano B para falhas.

Critérios de aceitação:

- Os slides contam problema, solução, arquitetura, evidência da integração, resultados e próximos passos dentro do tempo previsto.
- O roteiro da demonstração mostra consulta de linha, itinerário, previsão e uma falha com nova tentativa.
- Existe plano B se a API ou a internet falharem (vídeo gravado e dados de fixture).

Sub-issues:

- **ISS-41.1 — Slides.** *Aceite:* versão revisada pelo grupo.

- **ISS-41.2 — Roteiro da demonstração e divisão de falas.** *Aceite:* roteiro com tempos por pessoa.
- **ISS-41.3 — Plano B:** gravação da demonstração e, se necessário, um modo de demonstração com dados de fixture, ativado por variável de ambiente. **Depende de:** ISS-41.2. *Aceite:* demonstração roda sem internet.

ISS-42 — Ensaio geral e congelamento

Milestone: E5 · **Prioridade:** Must · **Depende de:** ISS-41

O que fazer: ensaiar com cronômetro no ambiente final, congelar o código três dias antes e marcar a versão final.

Critérios de aceitação:

- Pelo menos um ensaio completo, com cronômetro, no equipamento da apresentação.
- A partir do congelamento só entram correções críticas.
- Tag `v1.0.0` na `main` e README final atualizado.

Sub-issues:

- **ISS-42.1 — Ensaio geral e ajustes.** *Aceite:* ensaio dentro do tempo.
- **ISS-42.2 — Congelar o código e criar `v1.0.0`. Depende de:** ISS-42.1. *Aceite:* tag criada e README conferido.

Fase 8 — Opcionais (Could Have, só se houver tempo)

Entram apenas com o MVP, a qualidade e a Entrega 5 sob controle, e antes do congelamento da ISS-42.2.

ISS-43 — Pontos de referência da linha (US13)

Milestone: E5 · **Prioridade:** Could · **Origem:** US13 / F13 · **Depende de:** ISS-17

O que fazer: destacar parques, shoppings e monumentos ao longo do itinerário. A SPTrans não fornece esse dado, então é preciso escolher a fonte.

Critérios de aceitação (da US13):

- Ao abrir o itinerário, os principais pontos de referência da rota aparecem destacados.
- A fonte dos dados e sua licença estão documentadas.

Sub-issues:

- **ISS-43.1 — Escolher a fonte** (lista curada à mão para as linhas do ponto ou dados do OpenStreetMap) e registrar em ADR. *Aceite:* decisão aprovada.

- **ISS-43.2 — Implementar a associação linha → pontos de referência e exibir na tela. Depende de:** ISS-43.1. *Aceite:* pontos aparecem no detalhe da linha.

ISS-44 — Sugestão de linhas por ponto de interesse (US14)

Milestone: E5 · **Prioridade:** Could · **Origem:** US14 / F14 · **Depende de:** ISS-43 e ISS-16

O que fazer: permitir buscar um destino ("Parque Ibirapuera") e sugerir linhas com paradas próximas.

Critérios de aceitação (da US14):

- Ao informar um ponto de interesse, o sistema devolve linhas com paradas próximas a ele.
- Sem resultado, a tela explica e sugere alternativas.

Sub-issues:

- **ISS-44.1 — Cadastro de pontos de interesse com coordenadas.** *Aceite:* lista inicial de destinos.
- **ISS-44.2 — Cálculo de proximidade entre pontos de interesse e paradas das linhas do ponto. Depende de:** ISS-44.1. *Aceite:* testes com distâncias conhecidas.
- **ISS-44.3 — Busca e resultado na interface. Depende de:** ISS-44.2. *Aceite:* fluxo completo demonstrável.

ISS-45 — Rotas multimodais (US15)

Milestone: Evolução futura · **Prioridade:** Could · **Origem:** US15 / F15 · **Depende de:** ISS-44

Recomendação: não implementar neste semestre. O primeiro incremento já exclui o planejamento multimodal, e ele exige outra fonte de dados (por exemplo, GTFS completo com um planejador de rotas). Manter a issue aberta no backlog como evolução futura, citada no plano de manutenção (ISS-35.2).

Critérios de aceitação (caso um dia seja feita):

- O trajeto aparece passo a passo, indicando os pontos de transferência.
- A fonte de dados e a viabilidade estão registradas em ADR antes de qualquer código.

Cobertura das histórias de usuário

História	Issues
US01 — Linhas e sentidos	ISS-03.3, ISS-06, ISS-08, ISS-15

História	Issues
US02 — Busca e seleção	ISS-08.2, ISS-16
US03 — Itinerário simplificado	ISS-10, ISS-17
US04 — Retorno por inatividade	ISS-21
US05 — Mapa do ponto	ISS-06.4, ISS-13.3, ISS-19
US06 — Previsão de chegada	ISS-04, ISS-07, ISS-09, ISS-18
US07 — Última atualização	ISS-07, ISS-09.3, ISS-18.4
US08 — Falhas e nova tentativa	ISS-05, ISS-09.2, ISS-13.2, ISS-20
US09 — Alto contraste	ISS-24
US10 — Sem depender de cor	ISS-25
US11 — Áudio	ISS-26
US12 — Comandos de voz	ISS-27
US13 — Pontos de referência	ISS-43
US14 — Linhas por ponto de interesse	ISS-44
US15 — Rotas multimodais	ISS-45 (não recomendada agora)